

PRD-01 Scope

Theme: Foundation + Authentication + User Management + Database + Project Architecture

Estimated Completion: 7–10 Days

PRODUCT REQUIREMENTS DOCUMENT (PRD-01)

Project

CSE STUDENT PLATFORM

Backend Foundation & Authentication System

Version: 1.0

1. Objective

Develop the complete backend foundation for the CSE Student Platform that provides a scalable, secure, and production-ready architecture.

This PRD establishes the core infrastructure that every future module will depend on, including authentication, user management, database design, API standards, validation, security, logging, and project architecture.

The implementation should be modular, maintainable, and suitable for future expansion without requiring significant refactoring.

2. Tech Stack

Runtime

- Node.js (Latest LTS)

Language

- TypeScript

Framework

- Express.js

Database

- Supabase PostgreSQL

ORM

- Prisma ORM

Authentication

- JWT
- Refresh Token
- bcrypt

Validation

- Zod

File Upload

- Multer

Logging

- Winston

Environment

- dotenv

API Documentation

- Swagger/OpenAPI
-

3. Project Architecture

Use a clean architecture with clear separation of concerns.

```
src/
├── config/
├── controllers/
├── services/
├── repositories/
├── routes/
├── middlewares/
├── validators/
├── prisma/
├── utils/
├── constants/
├── interfaces/
├── types/
├── helpers/
├── docs/
├── uploads/
├── logs/
├── app.ts
└── server.ts
```

The codebase must remain modular. Business logic should reside in services, database access in repositories, and controllers should only coordinate requests and responses.

4. Coding Standards

- Strict TypeScript mode enabled.
 - ESLint and Prettier configured.
 - Environment variables used for all secrets.
 - No hardcoded credentials.
 - Consistent API response format.
 - Centralized error handling.
 - Async/await used throughout.
 - Modular code with reusable utilities.
-

5. Database Design

Create the initial schema using Prisma.

User Table

Fields:

- id (UUID)
 - fullName
 - email
 - passwordHash
 - phoneNumber
 - profileImage
 - collegeName
 - university
 - branch
 - currentYear
 - semester
 - githubUrl
 - linkedinUrl
 - portfolioUrl
 - bio
 - role
 - isVerified
 - profileCompletion
 - createdAt
 - updatedAt
-

Refresh Token Table

Fields:

- id
 - userId
 - token
 - expiresAt
 - createdAt
-

Email Verification Table

Fields:

- id
 - email
 - otp
 - expiresAt
 - verified
-

Password Reset Table

Fields:

- id
 - email
 - otp
 - expiresAt
-

6. Authentication

Implement:

User Registration

Requirements:

- Validate input.
 - Email must be unique.
 - Password hashing using bcrypt.
 - Generate verification OTP.
 - Store verification request.
 - Return success response.
-

Email Verification

- Verify OTP.
 - Activate account.
 - Delete expired OTPs.
-

Login

Requirements:

- Email/password authentication.
 - JWT Access Token.
 - Refresh Token.
 - Secure cookie support (if applicable).
 - Record login time.
-

Refresh Token

- Issue new access token.

- Validate refresh token.
 - Revoke invalid tokens.
-

Logout

- Delete refresh token.
 - Invalidate active session.
-

Forgot Password

- Generate OTP.
 - Store securely.
 - Expire automatically.
-

Reset Password

- Validate OTP.
 - Update password hash.
 - Delete reset request.
 - Invalidate previous refresh tokens.
-

7. User Profile APIs

Authenticated users should be able to:

Get Profile

Returns:

- Basic Information
 - College Information
 - Social Links
 - Profile Completion
 - Account Status
-

Update Profile

Editable:

- Name
 - Phone
 - Bio
 - College
 - University
 - Branch
 - Semester
 - Year
 - GitHub
 - LinkedIn
 - Portfolio
-

Upload Profile Image

Store images using Supabase Storage.

Save the public URL in the database.

8. Middleware

Create reusable middleware.

Authentication Middleware

Validate JWT.

Attach authenticated user to the request.

Role Middleware

Support roles such as:

- Student
 - Mentor
 - Admin
-

Validation Middleware

Automatically validate request payloads using Zod.

Error Middleware

Global exception handling.

Never expose internal server errors to the client.

Logger Middleware

Log:

- Request method
 - Endpoint
 - Response time
 - Status code
 - IP address
-

9. Security

Implement:

- Password hashing
 - Helmet
 - CORS
 - Rate Limiting
 - Request Validation
 - SQL Injection protection (through Prisma)
 - XSS protection
 - Environment variable management
-

10. API Response Format

Every endpoint must return:

```
{
  "success": true,
  "message": "Description",
  "data": {},
  "errors": null
}
```

Error responses:

```
{
  "success": false,
  "message": "Validation Failed",
  "data": null,
  "errors": []
}
```

11. REST API Endpoints

Authentication

POST /api/auth/register

POST /api/auth/verify-email

POST /api/auth/login

POST /api/auth/logout

POST /api/auth/refresh

POST /api/auth/forgot-password

POST /api/auth/reset-password

Profile

GET /api/users/profile

PUT /api/users/profile

POST /api/users/profile-image

Health

GET /api/health

12. Logging

Store logs inside:

logs/

Log:

- Errors
 - Requests
 - Warnings
 - Startup
 - Database Connection
-

13. API Documentation

Configure Swagger.

Every endpoint should include:

- Description
 - Parameters
 - Request Body
 - Response Body
 - Authentication Requirement
 - Error Codes
-

14. Environment Variables

Support:

PORT

DATABASE_URL

DIRECT_URL

JWT_SECRET

JWT_REFRESH_SECRET

JWT_EXPIRY

REFRESH_EXPIRY

SUPABASE_URL

SUPABASE_ANON_KEY

SUPABASE_SERVICE_ROLE_KEY

SMTP_HOST

SMTP_PORT

SMTP_USER

SMTP_PASSWORD

CLIENT_URL

15. Acceptance Criteria

The implementation is considered complete when:

- Project builds successfully with no TypeScript errors.
 - Prisma connects successfully to Supabase PostgreSQL.
 - Database migrations run without issues.
 - User registration works.
 - Email verification works.
 - Login works.
 - JWT authentication works.
 - Refresh tokens work.
 - Password reset works.
 - Profile APIs work.
 - Profile image upload works.
 - Swagger documentation is available.
 - Logging is functional.
 - Security middleware is configured.
 - API responses follow the defined standard.
 - Code follows the prescribed architecture and coding standards.
-

Deliverables

By the end of PRD-01, the backend should provide a stable, secure, and extensible foundation for all future development. The project should be fully configured, documented, and ready for the next phase.